DESIGN AND ANALYSIS OF ALGORITHMS

5th Semester – Mini Project Report

NETWORK DESIGN OPTIMIZATION SYSTEM

Connecting a college campus with minimum cable cost using Kruskal's and Prim's algorithms

Table 1: Group members

#	Student Name	Enrollment No.
1	Mantra Joshi	12502080603010
2	Smit Rajput	12402080601132
3	Tanuj Rajpurohit	12402080601137
4	Prince Bhavnagariya	12402080601108

Submitted to: Yamini Dubey

Information Technology, ADIT, CVM

Academic Year 2026–27

Abstract

A college wants to connect all its buildings with network cable. There are many possible cable routes and each one has a different cost. Building every route would waste money, so we need the cheapest set of routes that still connects every building. In Design and Analysis of Algorithms this is called the **Minimum Spanning Tree (MST)** problem.

In this project we built a Python program that solves this problem with two greedy algorithms: **Kruskal's algorithm** and **Prim's algorithm**. The program shows every step of both algorithms, finds the minimum total cost, draws the final network, and compares the running time of both algorithms on networks of 10 to 2,000 buildings. For our sample campus of 8 buildings and 14 possible links, both algorithms give the same minimum cost of **249** instead of 597, a saving of 58.3%.

Keywords: Minimum Spanning Tree, Greedy algorithm, Kruskal, Prim, Union-Find, Min-heap.

Table of Contents

Abstract **2**

1. Introduction **4**

1.1 Problem Statement 4

1.2 Motivation and Real-World Use 4

1.3 Objectives 4

2. Problem Definition **4**

3. Existing Approach and Background **6**

4. Algorithm Design **6**

4.1 Kruskal's Algorithm 6

4.2 Prim's Algorithm 7

4.3 Why these two algorithms 8

5. Complexity Analysis **8**

6. Implementation **9**

7. Testing **10**

8. Performance Evaluation **10**

9. Screenshots **12**

10. Advantages, Limitations and Future Scope **13**

11. Conclusion **14**

References **14**

Appendix – Group Member Contributions **15**

1. Introduction

1.1 Problem Statement

Design a network that connects all buildings of a college campus with the **minimum total cable cost**, and compare two algorithms for doing this.

1.2 Motivation and Real-World Use

The same problem appears whenever many places must be connected cheaply:

- College and office networks (LAN cabling between buildings and labs)
- Telecom companies laying fibre cable between cities and mobile towers
- Electricity lines, water pipelines and gas pipelines
- Building roads that connect all villages at the lowest cost

Who benefits: colleges, companies and governments save money, and students learn how greedy algorithms solve a real problem.

1.3 Objectives

1. Model the campus network as a weighted graph.
2. Implement Kruskal's and Prim's algorithms in Python.
3. Show every step of both algorithms.
4. Find the minimum cost and the money saved compared with building every link.
5. Compare the running time of both algorithms for 10 to 2,000 buildings.
6. Analyse the best, average and worst-case time complexity and the space complexity.

2. Problem Definition

We represent the campus as a graph **G = (V, E)**: every building is a **vertex**, every possible cable route is an **edge**, and the cable cost is the **weight** of the edge.

Table 2: Problem definition

Item	Details
Input	List of buildings and list of possible links (building 1, building 2, cost)
Output	Links to build, minimum total cost, number of links, money saved
Constraints	Every building must be connected; there must be no cycle (loop); the answer has exactly V − 1 links
Objective	Make the total cost of the selected links as small as possible

Table 3: Sample input – campus with 8 buildings and 14 possible links (cost in Rs. thousand)

#	Building 1	Building 2	Cost
1	Admin	Library	40
2	Admin	Auditorium	35
3	Admin	Canteen	55
4	Library	CSE Lab	30
5	Library	Canteen	39
6	CSE Lab	IT Lab	25
7	CSE Lab	Canteen	50

8	IT Lab	Hostel	45
9	IT Lab	Canteen	60
10	Canteen	Hostel	50
11	Canteen	Auditorium	38
12	Canteen	Sports Ground	42
13	Auditorium	Sports Ground	48
14	Hostel	Sports Ground	40

If we build all 14 links the cost is **597**.

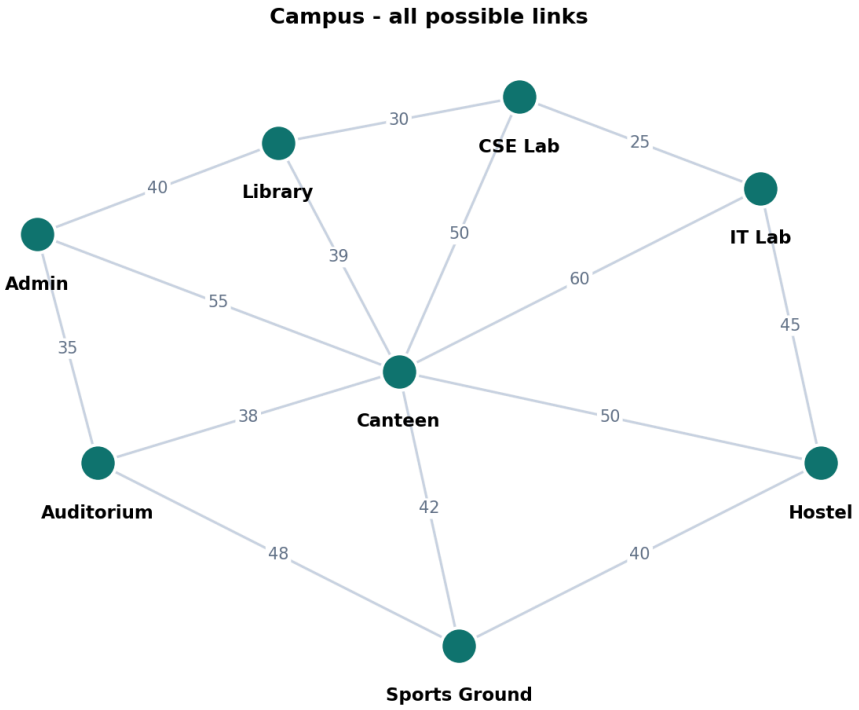


Figure 1: Campus network with all possible links

3. Existing Approach and Background

Table 4: Simple approaches and their problems

Approach	Problem
Build every link	Very expensive (cost 597)
Choose links by guess	No guarantee that it is the cheapest
Try every combination (brute force)	Too slow: for 8 buildings there are C(14, 7) = 3,432 combinations, and the number grows exponentially

Why a greedy approach works here: for the MST problem it has been proved that the cheapest link that joins two separate parts of the network is always part of some minimum spanning tree (the “cut property”). So always picking the cheapest safe link gives the best answer. Kruskal (1956) and Prim (1957) are the two standard greedy algorithms for this.

4. Algorithm Design

4.1 Kruskal's Algorithm

Idea: sort all links from cheapest to most expensive. Take each link in order. If the two buildings are not yet connected, accept the link. If they are already connected, reject it because it would make a cycle. Stop when V − 1 links have been accepted.

To check quickly whether two buildings are already connected we use **Union-Find (Disjoint Set)**. FIND gives the group of a building; UNION joins two groups.

```
KRUSKAL(buildings, links):
    sort links by cost (smallest first)
    put every building in its own group
    network = empty
    for each link (a, b, cost) in sorted order:
        if FIND(a) != FIND(b):      // different groups
            add the link to network
            UNION(a, b)             // join the two groups
        if network has V - 1 links: stop
    return network
```

Table 5: Kruskal's algorithm on the campus, step by step

Step	Link	Cost	Action	Total so far
1	CSE Lab – IT Lab	25	ACCEPT	25
2	Library – CSE Lab	30	ACCEPT	55
3	Admin – Auditorium	35	ACCEPT	90
4	Canteen – Auditorium	38	ACCEPT	128
5	Library – Canteen	39	ACCEPT	167
6	Admin – Library	40	REJECT (cycle)	167
7	Hostel – Sports Ground	40	ACCEPT	207
8	Canteen – Sports Ground	42	ACCEPT	249

Step 6 (Admin – Library) is rejected because Admin and Library are already connected through Auditorium, Canteen and Library. The final cost is **249** with 7 links.

4.2 Prim's Algorithm

Idea: start from one building (Admin). Look at all links that go from the buildings already in the network to buildings not yet in it, and add the cheapest one. Repeat until every building is in the network. A **min-heap (priority queue)** gives us the cheapest link quickly.

```
PRIM(buildings, links, start):
    network_buildings = {start}
    heap = all links of start
    while heap is not empty and not all buildings added:
        take the cheapest link (a, b) from heap
        if b is already in network_buildings: skip it
        add the link, add b to network_buildings
        push all links of b into heap
    return network
```

Table 6: Prim's algorithm on the campus (start = Admin)

Step	Link	Cost	New building	Total so far
1	Admin – Auditorium	35	Auditorium	35
2	Auditorium – Canteen	38	Canteen	73
3	Canteen – Library	39	Library	112
4	Library – CSE Lab	30	CSE Lab	142
5	CSE Lab – IT Lab	25	IT Lab	167
6	Canteen – Sports Ground	42	Sports Ground	209
7	Sports Ground – Hostel	40	Hostel	249

Prim picks the links in a different order, but the final cost is the same: **249**.

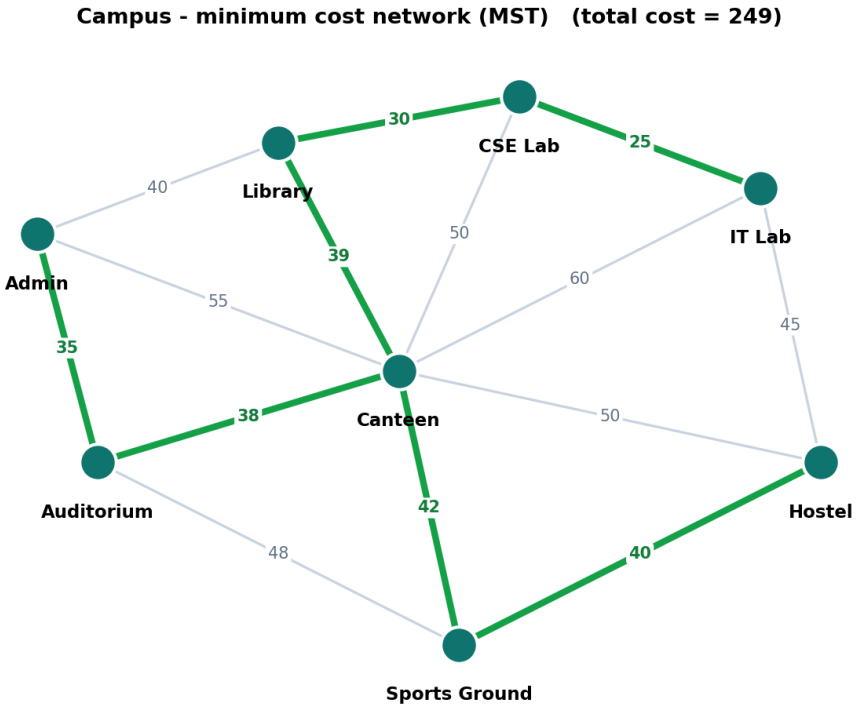


Figure 2: Final minimum cost network (green links)

4.3 Why these two algorithms

Table 7: Comparison of the two algorithms

Point	Kruskal	Prim
Works on	List of links (edges)	Buildings and their neighbours
Data structure	Union-Find	Min-heap (priority queue)
Best for	Networks with few links (sparse)	Networks with many links (dense)
Technique	Greedy + sorting	Greedy + priority queue

5. Complexity Analysis

V = number of buildings, E = number of links.

- **Kruskal:** sorting E links takes O(E log E). Each FIND / UNION is almost constant time with path compression, so the loop takes about O(E). Total: **O(E log E)**. The sort is always done, so best, average and worst case are all O(E log E).
- **Prim:** every link is pushed into the heap at most twice and each push / pop takes O(log E) = O(log V). Total: **O(E log V)** in every case.
- **Space:** both store the buildings and links, so both need **O(V + E)** memory.
- **Brute force** (for comparison) checks every combination of V − 1 links, which is exponential.

Table 8: Complexity table

Algorithm	Best case	Average case	Worst case	Space
Kruskal	O(E log E)	O(E log E)	O(E log E)	O(V + E)
Prim (min-heap)	O(E log V)	O(E log V)	O(E log V)	O(V + E)

6. Implementation

Language: Python 3. **Libraries:** heapq (built into Python, used for the min-heap), random and time (built-in), csv (built-in), Matplotlib (to draw the network picture and the performance graph), unittest (built-in, for testing).

Table 9: Main functions in network_design.py

Function	What it does
find(), union()	Union-Find: check and join groups of buildings
kruskal()	Kruskal's algorithm, prints each step if asked
prim()	Prim's algorithm using a min-heap
print_network(), print_result()	Show the input and the final answer
draw_network()	Save a picture of the network (all links and selected links)
random_network()	Create random networks of any size for testing
compare_performance()	Time both algorithms on 10 to 2,000 buildings, save table and graph
main()	Menu that lets the user choose what to run

How to run: open Terminal in the project folder and type python3 network_design.py . A menu appears; choose 1–6. To run the test cases type python3 test_network_design.py .

7. Testing

Table 10: Test cases (all passed)

#	Test	Expected result	Result
1	Sample campus (8 buildings, 14 links)	Cost 249 with 7 links for both	Pass
2	Triangle A-B (1), B-C (2), A-C (3)	Cost 3, link A-C left out	Pass
3	Only one building	Cost 0, no links	Pass
4	All links have the same cost	Cost 15 with 3 links	Pass
5	Two separate groups of buildings	Network cannot be fully connected	Pass
6	20 random networks (50–240 buildings)	Kruskal and Prim give the same cost	Pass

7	Large network (2,000 buildings)	Exactly 1,999 links selected	Pass

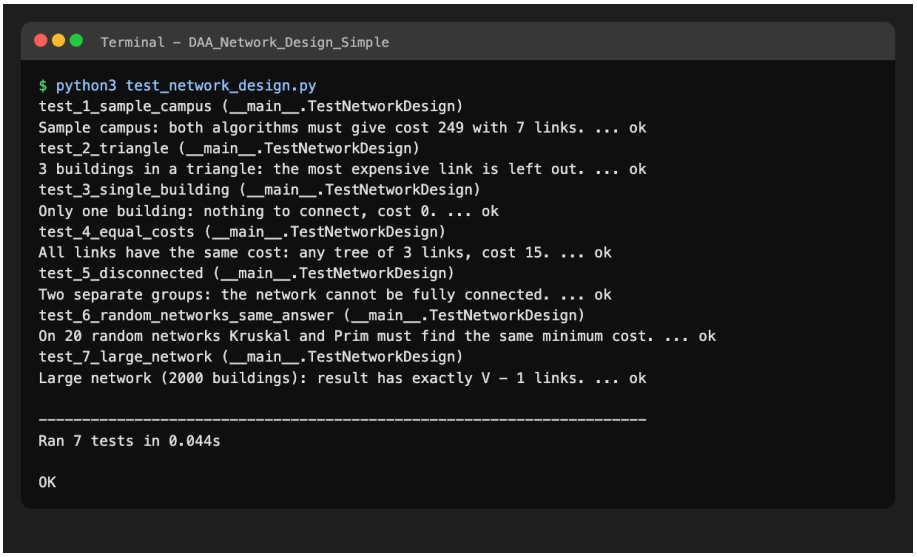


Figure 3: All 7 test cases passing

8. Performance Evaluation

We created random networks with 10 to 2,000 buildings (each building has about 4 links) and measured the average running time of each algorithm over 5 runs.

Table 11: Input size vs execution time

Buildings (V)	Links (E)	Kruskal (ms)	Prim (ms)	Same cost?
10	20	0.011	0.016	Yes
50	100	0.092	0.091	Yes
100	200	0.177	0.203	Yes
500	1000	1.075	1.141	Yes
1000	2000	2.247	2.635	Yes
2000	4000	4.695	6.008	Yes

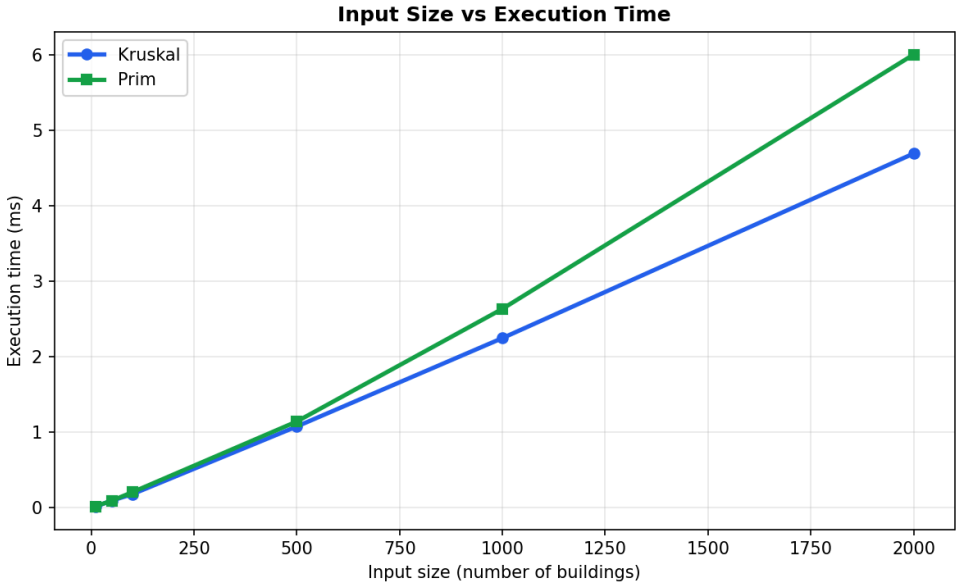


Figure 4: Input size vs execution time

Theory vs practice: when the number of buildings doubles from 500 to 1,000, Kruskal's time goes from 1.075 ms to 2.247 ms and Prim's from 1.141 ms to 2.635 ms – a little more than double, exactly as O(E log E) and O(E log V) predict (the log part makes it slightly more than 2×). Both algorithms always found the same minimum cost. In Python, Kruskal was slightly faster because Python's built-in sort is very fast.

9. Screenshots

```
Terminal – DAA_Network_Design_Simple

$ python3 network_design.py
============================================================
        NETWORK DESIGN OPTIMIZATION SYSTEM (MST)
============================================================
  1. Show campus network (input)
  2. Run Kruskal's algorithm (step by step)
  3. Run Prim's algorithm (step by step)
  4. Compare Kruskal vs Prim (performance test)
  5. Enter my own network
  6. Save pictures of the network
  0. Exit
  Enter your choice: 1

  Buildings (8): Admin, Library, CSE Lab, IT Lab, Hostel, Canteen, Auditorium, Sports Ground
  Possible cable links (14):
     Admin          – Library        cost  40
     Admin          – Auditorium     cost  35
     Admin          – Canteen        cost  55
     Library        – CSE Lab        cost  30
     Library        – Canteen        cost  39
     CSE Lab        – IT Lab         cost  25
     CSE Lab        – Canteen        cost  50
     IT Lab         – Hostel         cost  45
     IT Lab         – Canteen        cost  60
     Canteen        – Hostel         cost  50
     Canteen        – Auditorium     cost  38
     Canteen        – Sports Ground  cost  42
     Auditorium     – Sports Ground  cost  48
     Hostel         – Sports Ground  cost  40
  Cost if we build ALL links = 597
```

Figure 5: Input – menu and campus network

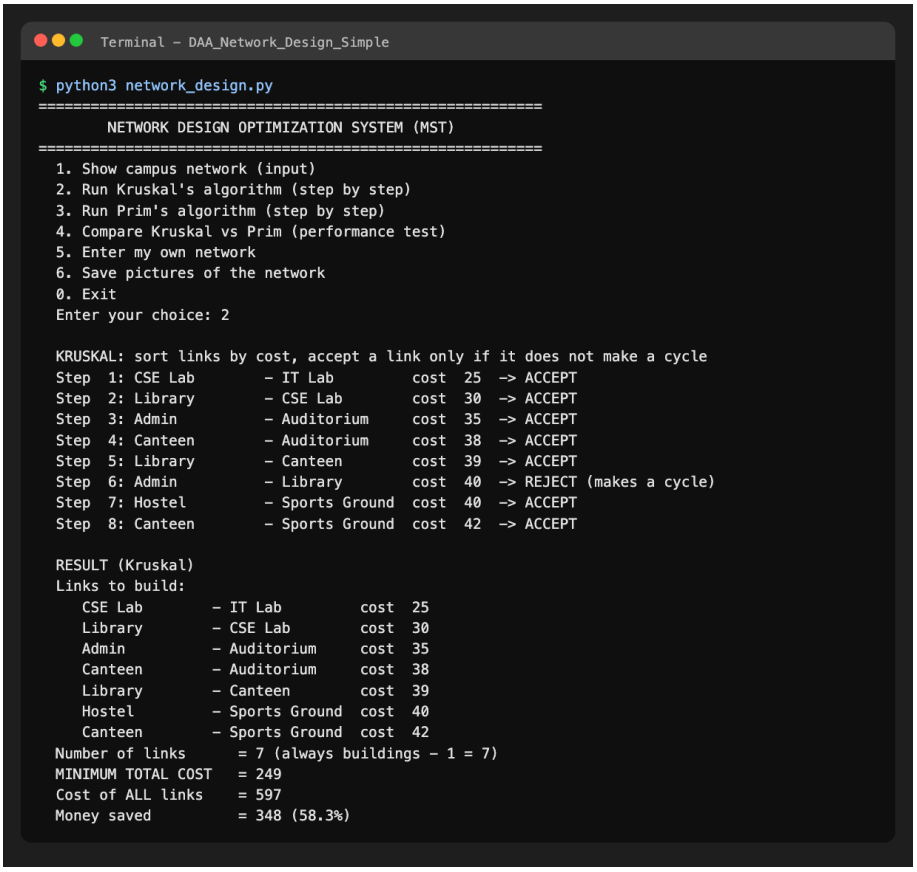


Figure 6: Processing – Kruskal's algorithm step by step

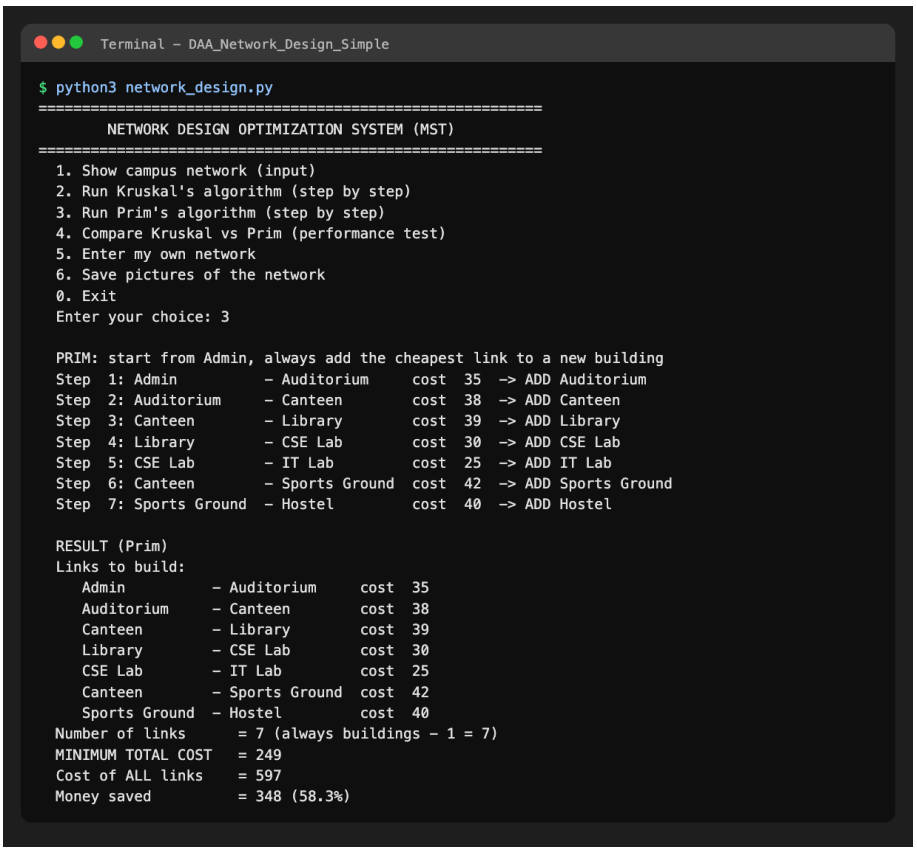


Figure 7: Processing – Prim's algorithm step by step

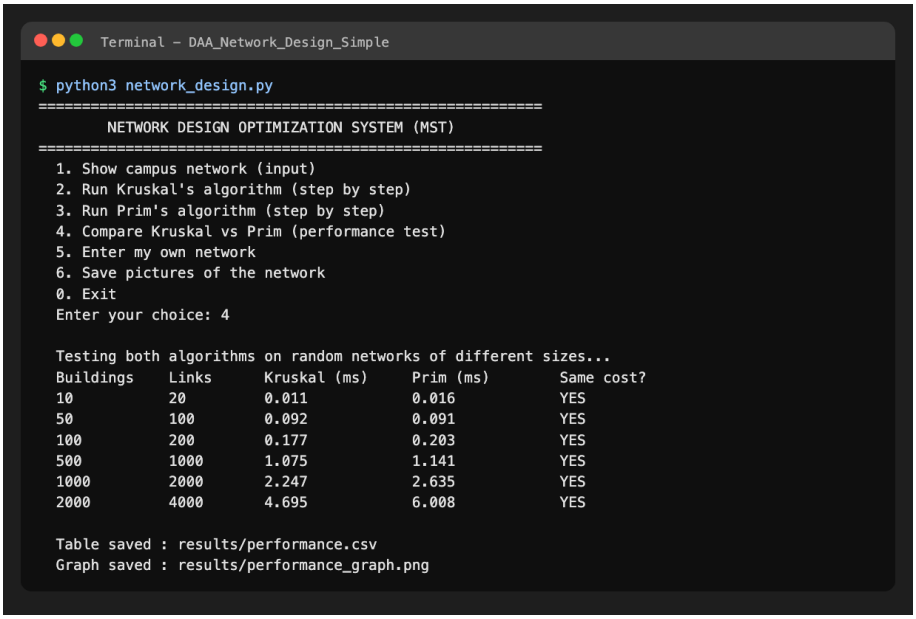


Figure 8: Performance comparison of both algorithms

10. Advantages, Limitations and Future Scope

Advantages

- Always gives the minimum possible cost.
- Very fast, even for thousands of buildings.
- Shows every step, so it is easy to understand.
- The user can enter their own network.

Limitations

- The result has only one path between two buildings, so one broken cable can disconnect part of the network.
- Costs are fixed estimates; real costs can change.
- Pictures are drawn only for the sample campus.

Future Scope

- Add backup links so the network keeps working if one cable breaks.
- Use real distances from a map.
- Add a graphical user interface.

11. Conclusion

We modelled campus network design as a Minimum Spanning Tree problem and solved it with Kruskal's and Prim's greedy algorithms. Both algorithms always gave the same minimum cost; for our campus the cost dropped from 597 to 249, a saving of 58.3%. The measured running times grew as predicted by the complexity analysis, and both algorithms handled 2,000 buildings in a few milliseconds.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, Introduction to Algorithms, 3rd/4th ed., MIT Press (Chapter: Minimum Spanning Trees).

[2] E. Horowitz, S. Sahni, S. Rajasekaran, Fundamentals of Computer Algorithms, Galgotia Publications.

[3] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem", Proceedings of the American Mathematical Society, 1956.

[4] R. C. Prim, "Shortest connection networks and some generalizations", Bell System Technical Journal, 1957.

[5] Python Software Foundation, heapq – Heap queue algorithm, https://docs.python.org/3/library/heapq.html

[6] Matplotlib documentation, https://matplotlib.org/

Appendix – Group Member Contributions

Table 12: Work division

Member	Responsibility
Mantra Joshi	Problem analysis and background study
Smit Rajput	Algorithm design and complexity analysis
Tanuj Rajpurohit	Implementation and testing
Prince Bhavnagariya	Results, screenshots, report and presentation